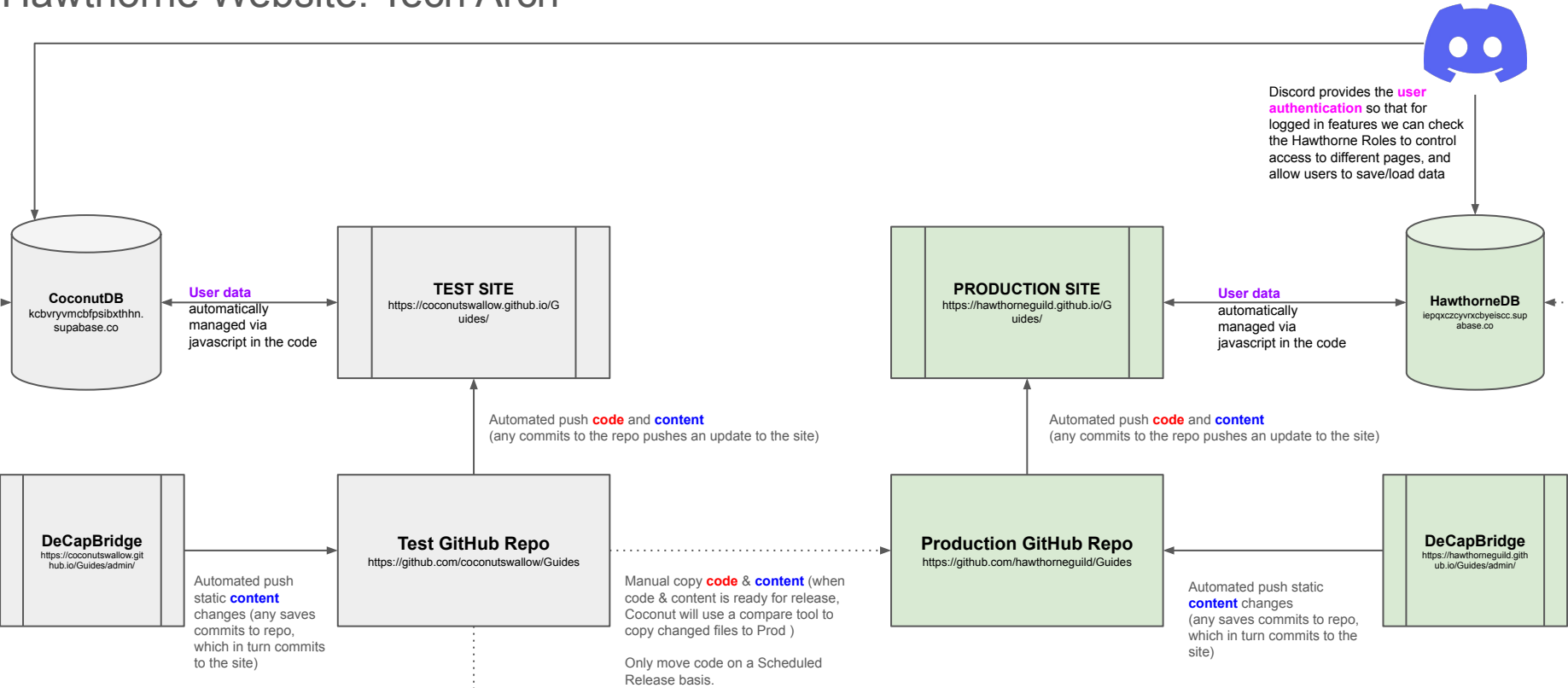
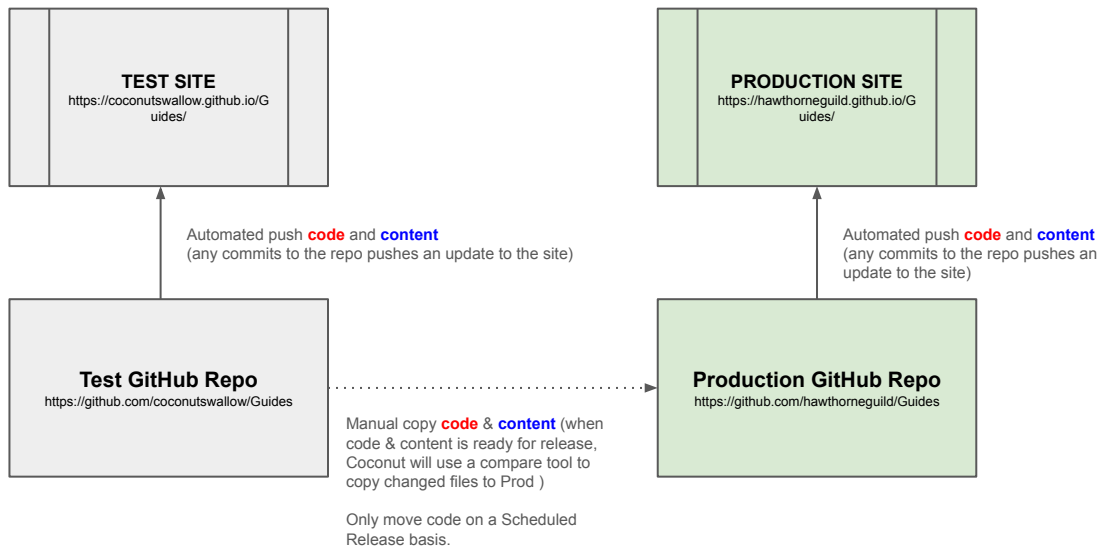


Hawthorne Website: Tech Arch

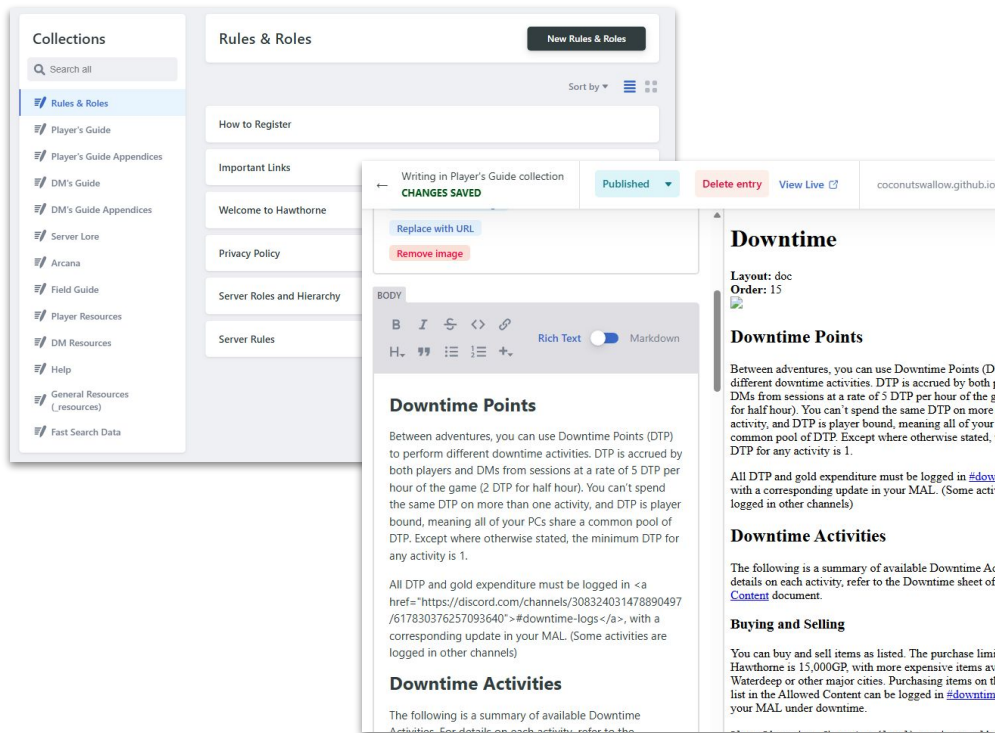


GitHub Pages



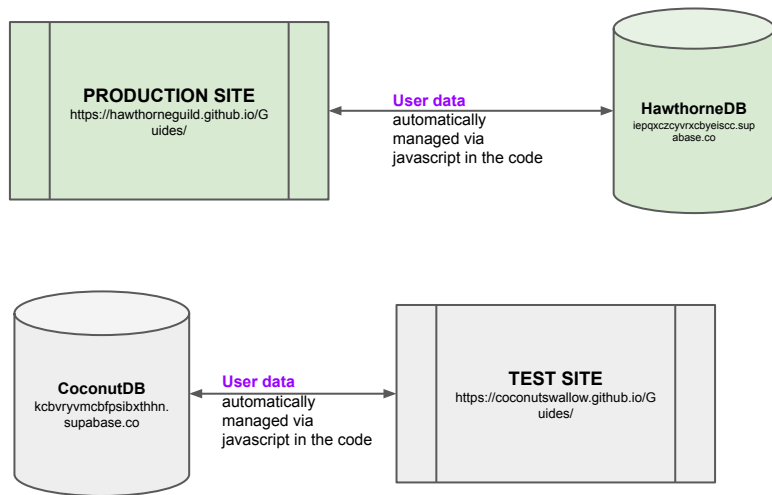
- Github pages is the foundation for the site. It used Jekyll CMS to render everything into static pages.
- Means it loads very fast, and has auto-indexing built-in
- It also has built-in feature to turn markdown (.md) into HTML using layouts/templates.
- Down-side is that technically has no server-side code. But we work around that with front-end javascript with a database
- The site is rebuilt **everytime a repo commit is made** (so i try not to do too many commits on the prod site)
- I have some additional documentation on how it works saved here: <https://hawthorneguild.github.io/Guides/help/>
- 2 environment "strategy"
 - My personal coconut repo I use for testing - allowing beta testers to see what's been built
 - Then when it's all tested and we're ready for a release,, I use a tool to sync the changes between the 2 repos.
 - Why do i have 2 sites that are manually sync'd instead of 1 site with branching? Because the pages are linked to repo commit - i can't figure out how to setup the branching strategy to render a second site with the same repo. It works, it's good enough.

DecapBridge



- DecapBridge is a tool that brings a “CMS” view to the site.
- This allows users to edit pages directly using a front-end UI
- The intention is in the future, staff can help maintain and add content (e.g. Lore can add new lore content themselves) without need for engineers
- This only works for “static” pages - basically content that are primarily text and converted from the PDF Guides
- Note: Any [save] immediately results in a commit to the repo -> which triggers a build of the site
- An admin needs to add a user before they can have access to make changes:
 - Coconut manages the admin for the dev version (<https://coconutswallow.github.io/Guides/admin/>) and
 - Luolang manages the admin for production site (<https://hawthorneguild.github.io/Guides/admin/>)

Supabase



Supabase is the cloud SQL database. There are a number of javascript modules in the site that interact with the database:

- Discord Authentication (see page on that)
- Monster Compendium
- DM Tool
- Rework Tool
- Allowed Content

I originally had just 1 database (HawthorneDB), but realized that i couldn't test anything in Prod, so i repurposed the database i used for my Codex for a Test database (you can only have 2 databases under the free plan)

To keep them in sync, at least structurally, any database change (table add/modify/etc.) I save the SQL as a .sql file in `/supabase/migrations` in the repo.

This way, if i make a change in dev, when we're ready, I just re-run the SQL changes to prod.

The routing to the database is controlled by `/assets/js/SupabaseClient.js`, which has code to detect which database to connect to in Test or Prod.

Discord Login

It's all documented here:

<https://github.com/hawthorneguild/HawthorneTeams/issues/17>

Basically included in supabase is built in oauth connection to discord, and i've set it up so that when you login, it asks for permissions to look at your guilds so it can get your assigned roles for Hawthorne.

This allows me to check if you're staff (for admin access) or if you're a DM (access to the DM tool, monster maker, etc.)